

个人网站平台产品需求文档 PRD

执行摘要

本 PRD 面向一个“个人网站/个人门户”产品：以你本人为主要使用对象，兼顾熟人（受邀）与访客（只读/受限访问），集中承载一组个人开发与运维相关能力（云盘、测速、frp、HFS 临时传输、SSH 到个人设备、Minecraft 服务器、邮件、远程开发、个人博客、Wiki 等），并预留将来持续扩展更多模块的能力（可插拔、可配置、可审计）。整体技术栈确定为 Python (Django) + PostgreSQL + Vue，视觉与交互遵循 Fluent 2 Design System，以统一的导航、认证与权限、日志审计、运维可观测性与备份恢复体系，降低“工具分散、入口杂乱、权限不可控、维护成本高”的问题。Fluent 2 的设计 token（颜色/字体/间距/阴影等）与排版、布局体系将作为全站 UI 规范的基础。 ¹

关键工程策略如下：

前端使用 Vue SPA（建议 Vue Router + Pinia）并与 Fluent UI Web Components 的 token/组件体系适配；后端以 Django 提供站点与 API（建议配合 Django REST framework），通过 session+CSRF（或可选 token/JWT）满足 SPA 调用安全；涉及终端/实时状态（SSH、任务、服务日志）使用 ASGI + WebSocket (Django Channels) 实现；数据层以 PostgreSQL 存放业务数据及检索索引（可启用全文检索 tsvector），文件类数据采用对象/文件存储（不入库，仅存 metadata）；运维层由统一反向代理终结 TLS (ACME/Let's Encrypt)，容器化部署 (Docker Compose) 并引入健康检查、监控与日志聚合；对外暴露服务时优先“反向代理 + 访问控制 + 速率限制 + (可选) WAF”，必要时结合 frp 做内网穿透。 ²

背景、目标与范围

背景

你正在规划一个个人网站，目标并非“单一网站内容展示”，而是一个自托管 (self-hosted) 平台入口：集成自用工具、服务器能力与知识沉淀内容，并在同一套身份体系与权限边界内向熟人/访客开放部分能力（例如只读博客、可控的临时分享链接、受限的测速页面等）。

目标

- 统一入口**：所有模块在同域名/统一导航下可发现、可管理、可审计。
- 安全可控**：对外暴露能力最小化，默认不信任公网；做到 TLS、权限、操作审计、速率限制与分级访问。 ³
- 可维护**：模块化、可插拔，支持“添加新模块而不破坏既有模块”；运维标准化（备份/监控/更新/证书）。 ⁴
- 体验一致**：遵循 Fluent 2 的 token、排版与布局体系，保证跨模块一致的交互与视觉语言。 ⁵

范围边界与未指定项

- 域名、部署地点（家中/机房/VPS）、公网 IP、CDN/WAF 供应商、可用带宽与预算：**未指定**。
- OAuth/第三方登录：要求“支持本地账号”，OAuth 相关细节：**未指定**。
- 多租户：未明确多租户目标，默认按“单用户 + 受限访客模式”设计（详见后文）。

- 具体各服务（邮件、Minecraft、远程开发、HFS、frp）是“内置实现”还是“外部服务接入”：本文按“平台作为控制面 + 统一入口”来设计，具体实现方式可在开发期落地选型；未指定处均标注“未指定”。

全站架构与通用需求

系统架构草图

推荐采用“反向代理 + 前后端分离 + 控制面/数据面分离”的总体结构：Django/Vue 负责统一入口、权限、审计与配置；具体服务（如 code-server、Minecraft、HFS、邮件栈、LibreSpeed、frp）可作为容器/系统服务运行，由平台进行状态采集与受控操作（启动/停止/重载/生成配置等）。WebSocket 需求（SSH Web 终端、实时日志/任务通知）由 ASGI + Channels 支撑。⁶

flowchart LR

```
U[用户浏览器<br/>本人/熟人/访客] -->|HTTPS| RP[反向代理<br/>Nginx/Caddy(未指定)]
RP -->|静态资源| FE[Vue SPA<br/>Fluent2 风格]
RP -->|/api /admin| BE[Django 应用<br/>ASGI: Uvicorn/Gunicorn(未指定)]
RP -->|WebSocket| WS[Django Channels<br/>SSH/实时通知]
BE --> DB[(PostgreSQL)]
BE --> FS[(文件存储<br/>本地磁盘/对象存储 未指定)]
WS --> RL[(Redis: channel layer 可选)]
BE --> Q[(任务队列 Celery 可选)]
Q --> RL
```

subgraph Services[外部/旁路服务(可容器化)]

```
FRP[frp 客户端/服务端]
HFS[HFS 临时传输]
SPEED[LibreSpeed 测速]
DEV[code-server 远程开发]
MC[Minecraft Server]
MAIL[邮件栈 Postfix/Dovecot/Webmail(未指定)]
```

end

RP --- Services

BE --- Services

WS --- Services

- Nginx 反向代理 WebSocket 需要显式处理 Upgrade header；否则 SSH/Web 终端与部分开发服务的 WS 通道可能失败。⁷
- Django 运行在 ASGI 服务器上（如 Uvicorn），官方文档给出运行方式示例。⁸
- Channels 对 Django 做 WebSocket 扩展并基于 ASGI。⁹

Fluent 2 设计与前端规范

Fluent 2 强调用设计 token 表达颜色、字体、间距、阴影等基础要素，配合一致的排版与布局体系。全站建议以 token 为源，避免在 Vue 样式中硬编码像素/色值。⁵

建议前端技术与组件策略：

- Vue 端状态管理建议使用 Pinia；Vue 官方文档“状态管理”明确 Pinia 的官方推荐地位。¹⁰
- Fluent UI Web Components 支持与 Vue 等框架集成，并可利用 Fluent 风格的 design tokens。¹¹
- 组件库选型（Fluent UI Web Components vs Vue 原生组件实现）与主题切换策略（亮/暗/自定义）细节：**未指定**。

认证与授权方案

认证

- **本地账号**：使用 Django 自带认证系统（用户、密码散列、权限/组）作为基础。¹²
- **API 调用方式**：前后端分离下推荐两种模式（二选一或混用，具体：**未指定**）
- **Session + CSRF**：SPA 使用 HttpOnly session cookie，所有“unsafe method”（POST/PUT/PATCH/DELETE）携带 CSRF token；Django CSRF 机制与 DRF 的 SessionAuthentication 与 CSRF 关系需严格遵循官方说明。¹³
- **Token/JWT**：适合自动化脚本/个人工具；但需额外处理 token 生命周期、吊销与存储安全（细节：**未指定**）。
- **OAuth / OIDC**：需求未指定具体提供商；建议预留 OIDC 接入位。可参考 django-allauth 对 OpenID Connect provider 的配置方式；若将来需要自建 IdP，可评估 Django OAuth Toolkit 的 OIDC 支持（是否采用：**未指定**）。¹⁴

授权

- 采用 **RBAC（角色/组）+ 资源级 ACL** 的组合：
- 角色示例：Owner（你本人/超级管理员）、Trusted（熟人/受邀）、Visitor（访客/匿名或受限）。
- 资源级 ACL：对“文件/临时分享/博客草稿/Wiki 私有页/设备管理/服务器控制”等资源做细粒度授权。
- DRF 权限体系用于 API 访问控制（自定义 permission class + object permission）。¹⁵
- **未指定**：是否需要 MFA、多设备登录管理、账号注册/邀请流程等。

多用户/多租户场景说明

按当前描述更贴近“**单用户 + 受限访客模式**”：

- 平台主账号唯一（Owner），具备全部管理权限；熟人通过邀请获得部分权限；访客主要访问公开内容（例如博客/Wiki 的公开页、必要时的测速页）。
- 若未来要扩展为多用户/多租户：建议在核心资源表引入 `owner_user_id` 或 `tenant_id`，并将权限判断统一封装为“Policy 层”（可插拔），避免散落在各视图中（是否启用：**未指定**）。

数据与检索通用策略

- PostgreSQL 负责业务数据与索引；全文检索可使用 PostgreSQL `tsvector/tsquery`，并用 Django `SearchVector` 等接口维护索引。¹⁶
- 连接池（可选）：若并发连接较多可引入 PgBouncer，需理解其 `pool_mode` 与兼容性。¹⁷
- WebSocket/实时功能：Channels 的 channel layer 可配 `channels_redis`；需意识到 Redis Pub/Sub “at-most-once” 投递语义，重要事件应落库或可重放。¹⁸

备份与恢复策略

- PostgreSQL 备份策略建议至少覆盖：
- 逻辑备份（`pg_dump/pg_dumpall`）用于跨版本迁移/单库恢复；官方文档与中文资料均强调其适用场景与注意事项。¹⁹

- 物理备份 + WAL 归档实现 PITR (Continuous Archiving)，以达到点时间恢复能力。²⁰
- 工具建议（可选）：pgBackRest 提供相对完整的备份/恢复体验；可作为标准化方案参考（是否采用：未指定）。²¹
- 文件存储（云盘/附件/临时传输落地文件）应使用独立备份：快照/增量（rsync）、对象存储版本化等（具体：未指定）。
- 恢复演练：至少月度/季度做一次“从备份恢复到隔离环境”的演练（周期：未指定）。

日志与监控建议

- 指标：Prometheus 生态建议通过 exporter 采集三类指标：主机（node exporter 等）、数据库与应用。Prometheus 官方文档解释了 exporter 的定位。²²
- PostgreSQL 指标：可使用 postgres_exporter。²³
- 日志：Loki 作为日志聚合系统，文档描述其面向日志、仅索引标签而非全文的设计点。²⁴
- 建议全站统一：请求日志、审计日志（安全关键）、服务日志（frp/HFS/Minecraft 等）进入同一聚合链路（落地方案：未指定）。

CI/CD 与部署流水线建议

- 容器化：建议使用 Docker 镜像构建 + Docker Compose 编排；Compose 支持 healthcheck 定义，且可覆盖 Dockerfile HEALTHCHECK。²⁵
- 镜像策略（建议）：
- app-web (Django ASGI)、app-worker (Celery worker 可选)、app-beat (Celery beat 可选)、nginx (反向代理)、postgres、以及各服务容器（code-server、LibreSpeed、Minecraft、mail stack 等）。
- Tag: <git_sha> + latest (或 stable)，并保留最近 N 个版本（N：未指定）。
- 健康检查：对 /healthz、数据库连接、外部依赖（Redis、队列）使用轻量探针；健康检查的必要性与典型故障场景在中文资料中有较好总结（供参考）。²⁶
- 部署方式：蓝绿/滚动（未指定）；个人站点可先采用停机短窗口的滚动替换。

性能与容量估算方法

建议采用“可测量指标驱动”的估算流程：

1. 统计资源类型：结构化数据（PostgreSQL）、文件数据（云盘/附件/临时传输）、日志/指标数据。
2. 估算请求模式：峰值并发用户、典型页面/接口 QPS、上传下载峰值带宽、长连接数（WebSocket）。
3. 以“模块-指标”方式拆解：例如云盘上传与下载、测速吞吐、Minecraft 在线人数、邮件每日收发量。
4. 从监控数据回填：上线后用 Prometheus 指标持续校准容量（采样周期：未指定）。²⁷

估算表格模板见后文“估算表格模板”。

安全合规要点

- TLS：全站必须 HTTPS；建议开启 HSTS，OWASP 说明了 HSTS 的作用与安全收益。²⁸
- 安全响应头：建议按 OWASP HTTP Headers Cheat Sheet 制定默认策略（CSP、X-Frame-Options、Referrer-Policy 等，具体值：未指定）。²⁹
- WAF/速率限制：
- 速率限制可用于保护登录、上传、管理接口等；Cloudflare 文档对 rate limiting 的用途与最佳实践有明确说明（是否采用 Cloudflare：未指定）。³⁰
- 如需自建 WAF，可评估 NGINX ModSecurity WAF 模块（F5 NGINX 文档）。³¹
- DDoS 缓解：建议至少具备“边缘阻断 + 速率限制 + 黑白名单 + 失败重试控制”的组合策略；Cloudflare 的 DDoS best practices 提供了可操作清单（供应商：未指定）。³²

- 邮件反垃圾/反伪造：SPF/DKIM/DMARC 的规范依据与导入方法建议参考权威指南（例如日本反垃圾相关组织的 DMARC 导入指南）。³³
- 邮件传输加固：MTA-STS 通过 HTTPS 发布策略以强制 SMTP TLS（RFC 8461），可增强抵抗降级攻击能力（是否启用：未指定）。³⁴
- SSH key 管理：必须支持 key-only（禁用密码/或至少对公网入口禁用），尽量禁 root 登录；暴力破解防护可引入 Fail2ban。³⁵
- 证书与私钥管理：ACME 客户端建议使用 Certbot（或等价 ACME 客户端）；Let's Encrypt 文档提供了客户端选项与推荐。³⁶

功能模块对照表

说明：API 端点为示例（最终以实现为准）；未指定的端点/字段可在设计阶段补齐。

模块名	简短描述	主要 API 端点示例	主要数据表（示例）
门户与系统管理	统一导航、仪表盘、全局搜索、健康状态、审计	<code>/api/v1/me</code> <code>/api/v1/health</code> <code>/api/v1/search</code> <code>/api/v1/audit-logs</code>	<code>user_profile</code> <code>feature_flag</code> <code>audit_log</code> <code>system_setting</code>
网盘（小）	个人文件管理与分享	<code>/api/v1/drive/items</code> <code>/api/v1/drive/upload</code> <code>/api/v1/drive/shares</code>	<code>drive_item</code> <code>file_blob</code> <code>share_link</code>
临时文件传输（HFS）	临时分享/外部 HFS 接入入口	<code>/api/v1/transfers</code> <code>/api/v1/transfers/{id}</code> <code>/t/{token}</code>	<code>temp_transfer</code> <code>temp_token</code> <code>transfer_event</code>
测速	内网/外网测速页面与结果留存	<code>/api/v1/speedtests</code> <code>/api/v1/speedtests/results</code>	<code>speedtest_result</code> <code>speedtest_session</code>
frp	frp 配置/状态/日志/重载	<code>/api/v1/frp/instances</code> <code>/api/v1/frp/proxies</code> <code>/api/v1/frp/reload</code>	<code>frp_instance</code> <code>frp_proxy</code> <code>config_revision</code> <code>service_log</code>
SSH（到个人设备）	Web 终端 + 设备清单 + 审计	<code>/api/v1/devices</code> <code>/ws/ssh/{device_id}</code>	<code>device</code> <code>ssh_identity</code> <code>ssh_session</code> <code>audit_log</code>
远程开发	code-server 等 IDE 的统一入口	<code>/api/v1/dev/workspaces</code> <code>/proxy/dev/{id}/...</code>	<code>dev_workspace</code> <code>proxy_route</code>
Minecraft 服务器	服务器状态、开关机、备份、白名单等	<code>/api/v1/mc/servers</code> <code>/api/v1/mc/servers/{id}/start</code> <code>/api/v1/mc/backups</code>	<code>mc_server</code> <code>mc_backup</code> <code>mc_player</code>
邮件	邮件账户/别名/域名配置入口 + Webmail	<code>/api/v1/mail/domains</code> <code>/api/v1/mail/users</code> <code>/api/v1/mail/aliases</code>	<code>mail_domain</code> <code>mail_user</code> <code>mail_alias</code>
个人博客	文章发布、标签、附件、评论（可选）	<code>/api/v1/blog/posts</code> <code>/api/v1/blog/tags</code>	<code>blog_post</code> <code>blog_tag</code> <code>content_attachment</code>

模块名	简短描述	主要 API 端点示例	主要数据表（示例）
Wiki	知识库、版本、权限、链接	<code>/api/v1/wiki/pages</code> <code>/api/v1/wiki/pages/{id}/revisions</code>	<code>wiki_page</code> <code>wiki_revision</code> <code>wiki_acl</code>

功能模块需求

通用约束：所有模块默认使用“单用户 + 受限访客”权限模型，未指定处标注“未指定”。所有管理类操作需落审计日志（可追溯）。设计风格统一为 Fluent 2（token + 排版 + 布局）。 5

门户与系统管理模块

功能概述

提供全站统一的入口与治理能力：模块导航、首页仪表盘（各服务状态/快捷入口）、全局搜索（跨博客/Wiki/文件名/配置）、系统设置、健康检查、审计日志查看、通知中心（如服务异常、备份失败）。全文检索建议优先使用 PostgreSQL 自带全文检索能力。 16

目标用户

你本人（Owner）、熟人（Trusted，部分可见）、访客（Visitor/匿名，只读）。

主要用例/用户故事

- 作为 Owner，我希望一打开首页就看到各服务运行状态与关键告警（例如 Minecraft 是否在线、邮件队列是否异常）。
- 作为 Trusted，我希望看到我被授权的入口（例如临时下载链接、Wiki 指定空间）。
- 作为访客，我只希望看到公开信息（博客/Wiki 公共页），不暴露管理入口。

关键功能点（前端交互与后端 API）

前端交互（Fluent 2）

- 左侧/顶部主导航（模块分组），可固定常用模块。
- 仪表盘卡片：服务状态（运行/停止/错误）、最近事件、快捷操作（Owner 可见）。
- 全局搜索框：统一搜索建议（autocomplete）、筛选（内容类型/标签/时间）。

后端 API（示例）

- `GET /api/v1/me`：当前用户信息、角色、可见模块。
- `GET /api/v1/health`：站点健康摘要（DB/缓存/队列/关键服务）。
- `GET /api/v1/search?q=`：跨模块搜索（结果按类型聚合）。
- `GET /api/v1/audit-logs?type=&actor=&time_range=`：审计日志查询。

数据模型草案（表结构要点）

- `system_setting`：key/value（加密字段：未指定）、生效范围（global/user）。
- `feature_flag`：模块开关、可见性策略（public/trusted/owner）。

- `audit_log` : `actor_user_id`、`action`、`target_type`、`target_id`、`ip`、`user_agent`、`created_at`、`extra_json`。
- `search_index` (可选) : 若不直接依赖 Postgres `tsvector`, 可用此表统一维护; 否则在各内容表新增 `search_vector` 字段 (实现细节: 未指定)。¹⁶

安全与权限考虑

- 管理入口默认需要 Owner 权限; Trusted 最小授权。
- 审计日志不可被非 Owner 清除; 导出能力 (是否提供: 未指定)。
- 全局搜索对受限内容必须做权限过滤 (避免 “通过搜索泄露标题/摘要”)。

运维/部署建议

- 反向代理统一 TLS; 开启 HSTS 与安全响应头。³⁷
- 监控: `/health` 探针、关键服务状态采集。
- 备份: 审计日志与配置应纳入 DB 备份范围 (PITR 或至少每日 `pg_dump`)。³⁸
- frp: 如站点部署在内网且需公网访问, 可通过 frp 暴露 443 并在 frp 层做 token/auth (要点见 frp 模块)。³⁹

与其他模块的依赖/集成点

- 依赖: 认证授权 (Django auth)、审计日志、搜索索引。
- 集成: 消费各服务模块的状态与事件流, 统一展示。

可扩展性与可插拔性建议

- 后端以 Django “app” 为模块边界: 每个模块提供 `register()` 元信息 (路由、权限、导航项、健康检查钩子)。
- 前端以 “动态路由 + 模块清单 (由 `/me` 返回)” 驱动导航, 实现模块按权限可见。
- 预留 “外部服务模块” 类型: 仅提供代理入口与健康检查, 不要求深度数据模型。

测试要点

- 权限矩阵测试: Owner/Trusted/Visitor 对导航、搜索、API 的访问差异。
- 安全测试: CSRF、越权访问、审计日志完整性。⁴⁰
- 健康检查: DB 断连、Redis 不可用、某服务容器停止时的降级表现。

云盘 (小) 模块

功能概述

提供小型个人云盘: 文件/文件夹管理、上传下载、基础预览 (图片/文本/视频: 未指定)、共享链接、权限控制、(可选) WebDAV。设计目标是 “够用、简单、安全、可控”, 并与博客/Wiki 的附件能力复用存储与权限体系。

目标用户

Owner (全功能); Trusted (按共享授权上传/下载/协作: 未指定); 访客 (仅可通过分享链接下载指定文件)。

主要用例/用户故事

- 我希望从浏览器上传文件到指定目录，并可生成分享链接发给熟人。
- 熟人通过链接下载文件，链接可设置过期时间与下载次数限制。
- 我希望能查看上传/下载历史，定位异常流量。

关键功能点（前端交互与后端 API）

前端交互

- 文件列表：支持排序（名称/时间/大小）、面包屑导航、右键/更多菜单。
- 上传：拖拽上传、断点续传（未指定）、上传进度、失败重试。
- 分享：一键生成链接，配置“过期时间/密码（未指定）/最大下载次数”。
- 预览：图片缩略图；文本预览（编码处理：未指定）。

后端 API（示例）

- GET /api/v1/drive/items?parent_id= : 列目录。
- POST /api/v1/drive/upload : multipart 上传；返回 file_id。
- GET /api/v1/drive/items/{id}/download : 下载（鉴权或 share token）。
- POST /api/v1/drive/shares : 创建分享链接。
- GET /t/drive/{token} : 分享下载入口（可匿名）。

数据模型草案（表结构要点）

- drive_item : id、parent_id、type (file/folder)、name、owner_user_id、acl_policy、created_at、updated_at。
- file_blob : id、drive_item_id、storage_backend (local/s3 未指定)、path、sha256、size、mime、created_at。
- share_link : token、resource_type、resource_id、expires_at、max_downloads、download_count、created_by。
- transfer_event (复用/共享) : 记录下载 IP、UA、时间。

安全与权限考虑

- 文件访问必须经权限校验（Owner/ACL/share token）。
- 分享 token 需足够随机；支持撤销。
- 上传内容安全：
- MIME/扩展名校验（策略：未指定）。
- 可选：病毒扫描（ClamAV 等，未指定）。
- 目录穿越与路径注入防护：存储路径不由用户输入直接拼接（实现细节：未指定）。

运维/部署建议

- 存储：文件落地目录需与应用容器分离（volume），并纳入备份策略。
- 备份：文件 + DB 元数据一致性（建议使用快照 + DB PITR 或在备份点冻结写入：方案未指定）。⁴¹
- 监控：上传/下载带宽、错误率、磁盘占用与 inode；异常下载告警。
- 证书/反向代理：统一由站点反向代理处理 TLS。⁴²
- 端口映射：仅暴露 443；内部文件服务走同一应用或内部对象存储端口（未指定）。
- frp：若通过 frp 暴露公网下载，建议对下载端点做速率限制与防盗链策略（未指定），并在 frp 与站点层都启用鉴权/访问控制。⁴³

与其他模块的依赖/集成点

- 依赖：统一权限与审计、存储抽象层。
- 集成：博客/Wiki 附件复用 `file_blob`；临时传输可复用 `share_link/token` 体系。

可扩展性与可插拔性建议

- 存储后端抽象：local filesystem 与 S3 兼容接口（具体：未指定）。
- 支持“目录级 ACL policy”与“外链策略”作为插件（例如一次性链接、仅允许某 IP 段下载等）。

测试要点

- 大文件上传/下载（超时、断点：若实现）。
- 权限绕过：直接访问 download URL；share token 过期/撤销。
- 并发下载下的带宽限制与资源占用。

临时文件传输（HFS）模块

功能概述

提供临时文件传输能力，满足“临时共享、无需进入云盘结构”的场景。实现形态分两种（择一或同时支持，未指定）：

1. **内置临时传输**：平台自己实现临时上传/生成一次性链接。
2. **HFS 接入**：外部运行 HFS（HTTP File Server），平台提供入口、鉴权代理、日志与开关。HFS 官方介绍其目标是通过 HTTP 共享磁盘文件，可对分享范围进行限制。 ⁴⁴

目标用户

Owner（创建/管理）；Trusted（按授权创建/下载：未指定）；访客（通过 token 下载或上传：未指定）。

主要用例/用户故事

- 我想把一个大文件临时发给朋友，链接 24 小时后自动失效。
- 我希望能限制下载速度/次数，并看到谁在下载。
- 若使用 HFS，我希望平台能一键启动/停止 HFS 并查看实时活动（HFS 支持活动监控与带宽控制能力）。 ⁴⁵

关键功能点（前端交互与后端 API）

前端交互

- 创建临时传输任务：上传文件或选择已有文件；设置过期时间、权限、限速（未指定）。
- 传输任务列表：状态（有效/过期/撤销）、访问统计、复制链接。
- 若 HFS 模式：显示 HFS 状态、活动连接、快速跳转到 HFS 前端（经代理）。

后端 API（示例）

- `POST /api/v1/transfers`：创建临时任务。
- `GET /api/v1/transfers`：查询列表。
- `POST /api/v1/transfers/{id}/revoke`：撤销。

- GET /t/{token} : 临时下载/上传入口。
- POST /api/v1/hfs/start|stop|restart (仅 HFS 接入模式, 权限: Owner)。

数据模型草案 (表结构要点)

- temp_transfer : id、type (upload/download)、file_blob_id、expires_at、created_by、status。
- temp_token : token、transfer_id、max_uses、used_count、created_at。
- transfer_event : 访问记录 (IP/UA/时间/结果码/字节数)。

安全与权限考虑

- token 随机性与撤销; 避免可枚举。
- 上传入口要防滥用: 速率限制、最大文件大小、可选验证码 (未指定)。 46
- 若代理 HFS:
- 不直接暴露 HFS 原端口到公网; 统一由反向代理与平台鉴权门控。
- 防止 HFS 目录暴露超出允许范围 (HFS 的虚拟文件系统功能与共享策略需严格配置)。 44

运维/部署建议

- 端口映射: HFS 容器/服务仅内网端口; 外部仅 443。
- 证书: 统一 TLS 终结于反向代理。 42
- 监控: 临时传输峰值带宽、失败率、存储占用; 过期清理任务 (定时任务实现: 未指定)。
- 备份: 临时传输一般不要求长期备份; 是否纳入备份: **未指定**。
- frp: 若通过 frp 暴露, 建议 HFS/临时下载路径单独限速与限流; 避免成为公网“免费中转站”。 47

与其他模块的依赖/集成点

- 可复用云盘的 file_blob 与 share_link/token; 也可独立存储。
- 与审计日志/告警联动 (异常下载告警)。

可扩展性与可插拔性建议

- 传输后端可插拔: 内置实现 / HFS / 未来其他工具 (如 Syncthing、Resilio: 未指定)。
- token 策略可插拔: 一次性、可刷新、绑定 IP、绑定邮箱 (未指定)。

测试要点

- token 过期/撤销行为; 并发下载计数一致性。
- 大文件下载中断续传 (若支持) 与 range 请求正确性。
- HFS 接入下的越权目录访问与代理头安全。

测速模块

功能概述

提供网络测速能力, 面向你自己与熟人: 测试到你站点/家庭网络的实际吞吐与延迟。建议采用 LibreSpeed (一类自托管 HTML5 speedtest), 其官方项目文档说明了可自建、可配置、多测试点等特性。 48

目标用户

Owner、Trusted（可使用并查看历史）；访客（是否开放：未指定）。

主要用例/用户故事

- 我希望从手机/电脑快速测试到家中网络的 download/upload/ping/jitter。
- 我希望测速结果可留存，便于对比不同时间段网络状况。
- 我希望能限制测速频率，避免被滥用导致带宽跑满。

关键功能点（前端交互与后端 API）

前端交互

- 测速页面嵌入 LibreSpeed UI（iframe 或直接集成其前端资源：未指定）。
- 显示历史记录图表（时间维度对比）。
- 权限控制：Trusted 才能看历史；公开页面只允许“开始测速”。

后端 API（示例）

- `POST /api/v1/speedtests/results`：提交一次测速结果（由前端回传或服务端汇总）。
- `GET /api/v1/speedtests/results?range=`：查询历史。
- `GET /api/v1/speedtests/policy`：频率限制与页面配置。

数据模型草案（表结构要点）

- `speedtest_session`：id、user_id/anonymous_token、started_at、client_ip。
- `speedtest_result`：session_id、ping_ms、jitter_ms、download_mbps、upload_mbps、packet_loss（未指定）、server_region（未指定）、created_at。

安全与权限考虑

- 测速接口必须限流；对匿名开放时更应强化（WAF/速率限制）。³⁰
- 客户端 IP/UA 等属于日志数据，是否展示给 Trusted：未指定（隐私边界要明确）。
- 防止被当作下载器：测速端点应固定路径与大小策略（未指定）。

运维/部署建议

- 端口：LibreSpeed 服务仅内网端口；反向代理暴露 `/tools/speedtest`。
- 证书：统一 TLS；HSTS 建议同域启用。⁴⁹
- 监控：带宽使用（峰值/平均）、错误率、请求量；异常峰值告警。
- 备份：测速结果属于低价值数据，备份策略：未指定。
- frp：若站点通过 frp 暴露公网，测速模块可能成为带宽瓶颈；建议按路径单独限流。⁴³

与其他模块的依赖/集成点

- 与门户仪表盘集成：展示近期均值与异常。
- 与审计/事件系统集成：识别滥用行为。

可扩展性与可插拔性建议

- 测速后端可替换为 iperf3（未指定）或多测试点模式。LibreSpeed 文档提到多服务器示例。⁵⁰
- 支持“测速节点管理”（未来扩展：未指定）。

测试要点

- 多浏览器/移动端兼容；弱网情况下 UI 表现。
- 限流规则是否生效。
- 历史数据写入与查询性能（索引）。

frp 模块

功能概述

提供 frp（内网穿透/反向代理）控制与可视化：管理 frp 实例（frpc/frps 的运行状态、配置模板、代理条目、日志、重载/验证）。frp 项目说明其可帮助将 NAT/防火墙后的服务暴露到公网，并支持多种协议；其文档也提到认证方式（默认 token 等）。⁵¹

目标用户

Owner（全功能）；Trusted（只读状态：未指定）；访客（不开放）。

主要用例/用户故事

- 我希望在网页上查看当前有哪些内网服务通过 frp 暴露、对应域名/端口、是否在线。
- 我希望修改 frpc 配置后先验证再重载，避免错误配置导致全站不可用（frp 支持配置校验命令）。⁵²
- 我希望轮换 token/认证方式，并记录变更历史。

关键功能点（前端交互与后端 API）

前端交互

- frp 实例卡片：版本、运行状态、连接数、最近心跳时间。
- 代理条目列表：name、type（tcp/http/https）、local_addr、remote_addr、domain、状态。
- 配置编辑器：语法高亮（toml/ini：未指定）、校验、提交、回滚。
- 日志查看：实时 tail（Owner），支持筛选。

后端 API（示例）

- GET /api/v1/frp/instances
- GET /api/v1/frp/instances/{id}/status
- PUT /api/v1/frp/instances/{id}/config
- POST /api/v1/frp/instances/{id}/verify（调用 frpc verify / frps verify）⁵²
- POST /api/v1/frp/instances/{id}/reload
- GET /api/v1/frp/instances/{id}/logs?follow=1

数据模型草案（表结构要点）

- frp_instance：id、mode（frpc/frps）、host、run_mode（systemd/docker）、current_config_rev、status。
- config_revision：id、instance_id、content（加密：未指定）、created_by、created_at、checksum。
- frp_proxy：按 UI 管理代理条目（可从 config 解析生成或独立管理：未指定）。
- service_log：日志聚合索引（或直接接 Loki：未指定）。⁵³

安全与权限考虑

- frp 管理属于高危操作：仅 Owner；所有变更必须审计。
- frp token/密钥：不得明文展示；存储方式（KMS/Vault/文件挂载：未指定）。
- 原则：frp 暴露端口应最小化，并与站点反向代理/WAF 配合（避免直接暴露内部管理服务）。 54

运维/部署建议（含 frp 配置要点）

- frp 建议容器化或 systemd 管理；提供“配置验证—重载”流程。 52
- 端口映射：
- frps 通常需要公网可达的 bindPort（如 7000，示例见官方配置范例）。 55
- 若通过 443/80 做 HTTP(S) 转发，需与反向代理域名/证书策略一致（细节：未指定）。
- 认证：frp 文档说明可配置 auth.method（默认 token）；应开启并定期轮换。 39
- 监控：连接数、心跳延迟、代理可用性探测（HTTP 200/端口连通）。
- 备份：frp 配置 revision 作为关键配置需进入 DB/配置仓库备份。 38

与其他模块的依赖/集成点

- 与“端口映射/反向代理”强耦合：暴露路径应在门户中登记。
- 为 SSH、远程开发、Minecraft 等模块提供公网入口（但应默认关闭，按需启用）。

可扩展性与可插拔性建议

- 支持多实例与环境（home/office/vps：未指定）。
- 支持“策略引擎”：对外暴露服务前必须满足安全检查（例如启用鉴权、限制来源 IP、开启 TLS）。

测试要点

- 配置校验与回滚。
- 重载过程中连接中断策略（可容忍：未指定）。
- 越权访问：Trusted 不得看到任何 secret 字段。

SSH（到个人设备）模块

功能概述

提供 Web 端 SSH 终端：在浏览器内连接到其他个人设备（NAS、树莓派、工作机等），并支持设备清单、快速连接、会话审计（至少记录元数据而非内容：未指定）。前端可采用 xterm.js 作为 Web 终端组件；其官方文档/仓库说明它可在浏览器提供完整终端能力。 56

后端实现建议：使用 Django Channels 提供 WebSocket 通道，把浏览器输入输出与 SSH 通道（paramiko/asyncssh/系统 ssh：未指定）桥接。 57

目标用户

Owner（全功能）；Trusted（是否允许 SSH：默认不允许，未指定）；访客（不允许）。

主要用例/用户故事

- 我希望不在本机保存复杂跳板配置，在网页里一键连到某台设备。
- 我希望能按设备分组，并看到最近连接时间。

- 我希望所有连接尝试与关键操作都有审计。

关键功能点（前端交互与后端 API）

前端交互

- 设备列表：名称、内网 IP/域名（展示策略：未指定）、标签、在线状态。
- 终端页：多标签会话、复制粘贴、快捷键提示、断线重连。
- 连接参数：用户名、端口、认证方式（key/agent：未指定）。

后端 API / WS（示例）

- GET /api/v1/devices
- POST /api/v1/devices（Owner 管理）
- POST /api/v1/ssh/sessions：创建会话，返回 session_id
- WS /ws/ssh/{session_id}：终端数据通道（Channels）⁵⁷

数据模型草案（表结构要点）

- device：id、name、host、port、tags、owner_user_id、last_seen_at（探测方式：未指定）。
- ssh_identity：引用密钥（仅保存 public key 与存储引用；私钥存储：未指定）。
- ssh_session：id、device_id、actor_user_id、started_at、ended_at、client_ip、result、extra_json。

安全与权限考虑

- 强制 key-based；禁用密码（至少对公网入口禁用）。⁵⁸
- 禁止 root 直登（策略：未指定），重要设备建议只允许跳板或 VPN（未指定）。
- 防暴力破解：对 SSH 服务端可用 Fail2ban 等手段；对 Web 端连接也要限流。⁵⁹
- 终端回显日志是否保存：**未指定**（风险高，涉及敏感信息）。

运维/部署建议

- WebSocket 反向代理必须正确配置 Upgrade（Nginx 官方说明）。⁷
- 端口：站点仅 443；SSH 目标端口在内网可达即可，尽量不直接公网暴露。
- 证书：统一 TLS；启用 HSTS。⁴⁹
- 监控：WebSocket 连接数、会话失败率、连接时延。
- frp：如必须从公网进来访问内网设备，优先通过站点（443）+ 受控 WebSSH，而不是直接把设备 22 暴露；若使用 frp 转发 22，需强认证与来源限制（未指定）。⁴³

与其他模块的依赖/集成点

- 依赖：账号权限、审计日志、WebSocket 基础设施。
- 集成：门户仪表盘显示“最近连接”“失败告警”。

可扩展性与可插拔性建议

- 连接后端可插拔（paramiko vs 系统 ssh）；
- 支持“设备能力插件”：例如一键执行健康检查脚本、拉取日志（未指定）。

测试要点

- WebSocket 稳定性、断线重连、并发会话。
- 权限：非 Owner 无法创建会话。
- 安全：命令注入/参数注入（host/port/user 输入校验）。

Minecraft 服务器模块

功能概述

提供 Minecraft 服务器的管理与使用入口：服务器状态查看、启停、版本/配置管理（部分未指定）、备份与恢复、白名单/OP 管理、日志查看、玩家在线列表等。官方 EULA 适用范围与需要接受 EULA 的流程在官方页面/帮助文档中均有说明；官方也提供了 Java 版服务器搭建步骤指导与专门的 Dedicated Server System Requirements（最低配置要求）。⁶⁰

目标用户

Owner（管理）；Trusted（作为玩家/查看状态）；访客（不开放或仅展示状态：未指定）。

主要用例/用户故事

- 我希望一键启动/停止服务器，并看到 CPU/RAM/在线人数。
- 我希望定期备份世界并可一键回滚/下载。
- 我希望白名单管理在网页完成，并记录谁改了白名单。

关键功能点（前端交互与后端 API）

前端交互

- 服务器概览：在线/离线、版本、在线人数、地图/世界名（未指定）、最近备份。
- 操作按钮：启动/停止/重启（Owner）、查看日志、下载备份。
- 白名单：添加/删除玩家 ID、导入导出（未指定）。

后端 API（示例）

- GET /api/v1/mc/servers
- POST /api/v1/mc/servers/{id}/start|stop|restart
- GET /api/v1/mc/servers/{id}/status （在线人数/资源占用）
- POST /api/v1/mc/servers/{id}/backups （触发备份）
- GET /api/v1/mc/backups/{backup_id}/download

数据模型草案（表结构要点）

- mc_server：id、name、runtime (docker/systemd)、version、port、status、created_at。
- mc_backup：id、server_id、type (full/incremental 未指定)、size、path、created_at、created_by。
- mc_player：server_id、player_name、uuid（来源：未指定）、whitelisted、op_level。

安全与权限考虑

- 服务器控制 API 仅 Owner。
- 日志可能含 IP/聊天内容，展示范围与脱敏策略：未指定。
- 若提供 RCON：必须强密码并仅内网可达，且通过平台代理访问（细节：未指定）。

运维/部署建议

- 运行方式：建议容器化，便于资源限制与升级回滚（具体镜像：未指定）。
- 端口：游戏端口对外开放策略（公网/内网）未指定；若通过 frp 暴露，需与站点分离，避免影响主站。³⁹
- 备份：世界数据属于高价值，应纳入备份；可结合 Postgres 的 PITR 方案做统一灾备演练。⁴¹
- 监控：在线人数、TPS（如可采集，未指定）、资源占用、磁盘增长。
- 法务/合规：遵循 EULA，并在管理界面显式提示“需接受 EULA”。⁶¹

与其他模块的依赖/集成点

- 门户仪表盘展示状态；审计日志记录启停/白名单修改。
- 云盘/临时传输用于分发备份文件（可选）。

可扩展性与可插拔性建议

- 支持多个服务器实例（原版/模组/不同版本）；
- 管理后端可插拔（systemd、Docker、K8s：未指定）。

测试要点

- 启停操作幂等性；并发操作冲突（重复点击）。
- 备份一致性：在线备份是否需要停服（未指定）。
- 权限与审计：所有高危操作必须记录。

邮件模块

功能概述

提供邮件相关能力的“统一入口 +（可选）控制面”：

- 若你自建邮件服务器：在平台中管理域名、账户、别名、TLS/反垃圾配置状态，并提供 Webmail 入口（具体 Webmail：未指定）。
- 若邮件托管在第三方：平台提供只读状态与跳转入口（未指定）。

邮件栈相关权威资料：Postfix 官方文档含 TLS 支持细节；Dovecot 官方文档描述 SSL/TLS 配置；DMARC/SPF/DKIM 的规范依据可参考权威指南。⁶²

目标用户

Owner（管理）；Trusted（仅邮箱使用入口，管理权限：未指定）；访客（不开放）。

主要用例/用户故事

- 我希望在网页上管理 `@mydomain` 的别名与转发规则，并一键查看 DNS 需要配置的记录。

- 我希望能看到“SPF/DKIM/DMARC 是否配置正确”的状态提示。
- 我希望邮件服务异常时能在门户收到告警。

关键功能点（前端交互与后端 API）

前端交互

- 域名卡片：MX、SPF、DKIM、DMARC、MTA-STS（可选）状态检测（检测方式：未指定）。
- 用户与别名：列表、创建、禁用、转发目标管理。
- Webmail：统一入口（iframe/跳转，未指定）。

后端 API（示例）

- GET /api/v1/mail/domains
- POST /api/v1/mail/domains
- GET /api/v1/mail/users POST /api/v1/mail/users
- GET /api/v1/mail/aliases POST /api/v1/mail/aliases
- GET /api/v1/mail/security-status（聚合 SPF/DKIM/DMARC/MTA-STS 检测结果）

数据模型草案（表结构要点）

- mail_domain：domain、status、created_at。
- mail_user：domain_id、local_part、password_hash（算法：未指定）或 external_ref、enabled。
- mail_alias：source、targets（json/多表，未指定）、enabled。
- mail_security_check：缓存检测结果与时间戳（TTL：未指定）。

安全与权限考虑

- 邮件账户/密码管理属于高敏：建议仅 Owner，或极少数 Trusted。
- TLS：Postfix/Dovecot 均需启用 TLS；Postfix 的 TLS_README 是重要参考。⁶³
- 反伪造：SPF/DKIM/DMARC 建议按权威指南逐步部署，并先观察报告再收紧策略（策略细节：未指定）。³³
- 反垃圾：灰名单/内容过滤/RBL（未指定）。
- MTA-STS（可选）：参考 RFC 8461，若启用需 HTTPS 发布策略文件。⁶⁴

运维/部署建议

- 邮件服务端口（25/465/587/143/993 等）与证书管理复杂度高；建议明确“自建 vs 托管”边界（未指定）。
- 证书：Let's Encrypt/ACME；Let's Encrypt 推荐 Certbot（或等价工具）。³⁶
- 监控：队列长度、投递失败、TLS 握手失败、反垃圾命中率（未指定）。
- 备份：邮件存储（Maildir 等）与账号数据需备份；恢复窗口（RPO/RTO）未指定。
- frp：一般不建议用 frp 暴露 SMTP；应使用固定公网 IP 与合规的反垃圾策略（若必须：未指定）。

与其他模块的依赖/集成点

- 门户告警与仪表盘；审计日志记录域名/别名变更。
- OAuth/SSO：若将来希望统一登录 Webmail，可在认证模块扩展 OIDC（未指定）。⁶⁵

可扩展性与可插拔性建议

- 支持多域名；

- 将“邮件后端实现”抽象为 provider（Postfix/Dovecot、第三方 API：未指定）。

测试要点

- 权限与审计：确保无越权读取敏感字段。
- DNS 检测逻辑正确性（SPF 解析、DKIM selector：未指定）。
- 回归测试：变更别名后投递行为是否符合预期（端到端可用性测试）。

远程开发模块

功能概述

提供远程开发入口，推荐接入 code-server（浏览器版 VS Code）。code-server 文档提到其可代理端口并通过内置认证安全访问 web services。 ⁶⁶

平台职责定位：

- 工作区（workspace）管理（路径/容器/镜像、资源配额：未指定）
- 统一入口与权限
- 反向代理（路径或子域名）与 WebSocket 支持
- 状态与日志展示

目标用户

Owner（全功能）；Trusted（是否允许共享工作区：未指定）；访客（不开放）。

主要用例/用户故事

- 我希望在任何设备打开浏览器就能进入开发环境，并能访问内部服务。
- 我希望不同项目有不同工作区，并能限制资源。
- 我希望 IDE 入口在平台侧统一权限与审计。

关键功能点（前端交互与后端 API）

前端交互

- 工作区列表：名称、状态、最后打开时间、资源占用（未指定）。
- 一键进入：点击后经平台代理到 code-server。
- 快捷配置：创建工作区时选择镜像、挂载路径、Git 远程（未指定）。

后端 API（示例）

- GET /api/v1/dev/workspaces
- POST /api/v1/dev/workspaces
- POST /api/v1/dev/workspaces/{id}/start|stop
- GET /proxy/dev/{id}/...：反代到 code-server（需 WS 支持）

数据模型草案（表结构要点）

- dev_workspace：id、name、runtime (docker)、image、mounts（未指定）、status、created_at。

- `proxy_route` : `workspace_id`、`base_path/subdomain`（未指定）、`target`。

安全与权限考虑

- 远程开发环境权限极高：默认仅 Owner。
- 必须防止 SSRF/路径穿越：代理目标地址必须来自白名单配置。
- WebSocket 反代配置正确性，否则 IDE 功能不完整（Nginx WebSocket proxying 官方说明）。 7

运维/部署建议

- code-server 容器化；仅内网端口；由反向代理统一 TLS。 42
- 端口代理：code-server 文档提到可代理端口（用于访问工作区内 web 服务），但需明确平台代理与 code-server 自带代理的边界（未指定）。 66
- 监控：工作区资源（CPU/RAM/磁盘）、WebSocket 连接数、登录失败。
- 备份：工作区数据（代码）是否备份取决于 Git 远程与存储策略（未指定）。
- frp：若站点在内网，外网开发接入可通过 frp 暴露 443；避免直接暴露 code-server 端口。 39

与其他模块的依赖/集成点

- 与 SSH 模块互补：一个偏终端，一个偏 IDE。
- 与云盘模块：共享附件/构建产物（可选）。

可扩展性与可插拔性建议

- 将工作区运行时抽象为 adapter（Docker、K8s、LXC：未指定）。
- 支持“只读共享工作区”给 Trusted（未来扩展，未指定）。

测试要点

- 反向代理 WS：打开/保存文件、终端等功能完整性。 7
- 权限：确保任何非 Owner 无法访问 `/proxy/dev/...`。
- 资源限制与逃逸风险（容器安全：未指定）。

个人博客模块

功能概述

提供个人博客：文章撰写（Markdown 或富文本：未指定）、发布/草稿、标签/分类、附件、评论（可选，未指定）、RSS（可选，未指定）、SEO（未指定）。对访客开放公开文章；对 Trusted 可开放私密文章（未指定）。

目标用户

Owner（创作/管理）；访客（阅读）；Trusted（阅读更多/评论：未指定）。

主要用例/用户故事

- 我希望用 Markdown 写作并预览，发布后生成永久链接。
- 我希望文章可设置公开/私密。
- 我希望全文搜索到文章内容。

关键功能点（前端交互与后端 API）

前端交互

- 编辑器（Markdown）：实时预览、图片拖拽上传（走云盘附件）。
- 文章页：目录（TOC）、代码高亮（未指定）。
- 列表页：分页、标签筛选。

后端 API（示例）

- GET /api/v1/blog/posts?status=published
- POST /api/v1/blog/posts (Owner)
- PUT /api/v1/blog/posts/{id}
- GET /blog/{slug} (SSR/静态化：未指定)
- GET /api/v1/search?q= (跨模块)

全文检索实现建议用 Postgres tsvector 与 Django SearchVector。 16

数据模型草案（表结构要点）

- blog_post：id、title、slug、content_md、content_html（是否存储：未指定）、status、visibility、published_at。
- blog_tag：name、slug。
- blog_post_tag：多对多。
- content_attachment：复用云盘 file_blob，并记录引用关系。
- blog_comment（可选，未指定）。

安全与权限考虑

- XSS：Markdown 渲染需做白名单过滤（具体库：未指定）；Django 安全文档强调不要信任用户输入。 67
- 管理 API 需要 CSRF（若 session auth）。 40
- 评论（若开放）：需防垃圾/限流（未指定）。 46

运维/部署建议

- 反向代理：公开页面可缓存（策略：未指定）。
- 备份：文章内容属于高价值；纳入 DB 备份与 PITR。 38
- 监控：访问量、404、渲染耗时。
- 证书：统一 TLS + HSTS。 49
- frp：若站点对外开放，博客是主要入口；建议配合 WAF/速率限制保护后台。 54

与其他模块的依赖/集成点

- 与云盘：附件上传。
- 与门户搜索：全文检索。
- 与审计：发布/删除记录。

可扩展性与可插拔性建议

- 支持静态化导出（未指定）。
- 支持多主题/暗色模式（基于 Fluent token，未指定）。 68

测试要点

- slug 唯一性与重定向。
- Markdown 渲染安全 (XSS) 。
- 权限：私密文章不可被匿名访问，也不能被搜索泄露摘要。

Wiki 模块

功能概述

提供个人 Wiki/知识库：页面树/空间（未指定）、Markdown 撰写、版本历史、页面间链接、标签、权限控制（公开/受邀/私有）、附件，支持全文检索。建议复用 Django + PostgreSQL 的全文检索能力。 16

目标用户

Owner（全功能）；Trusted（按空间/页面授权协作）；访客（可读公开页面）。

主要用例/用户故事

- 我希望把配置笔记、运维 SOP 记录到 Wiki 并可版本回滚。
- 我希望能快速引用其他页面并自动生成反向链接。
- 我希望熟人可协作编辑某一空间。

关键功能点（前端交互与后端 API）

前端交互

- Wiki 树/面包屑导航；页面内目录（TOC）；反向链接面板。
- 编辑：Markdown + 预览；保存时可填写变更说明（未指定）。
- 版本页：对比（diff）展示（实现方式：未指定）。

后端 API（示例）

- GET /api/v1/wiki/pages?parent_id=
- POST /api/v1/wiki/pages
- GET /api/v1/wiki/pages/{id}
- POST /api/v1/wiki/pages/{id}/revisions （创建新版本）
- GET /api/v1/wiki/pages/{id}/revisions
- POST /api/v1/wiki/pages/{id}/acl （Owner/空间管理员）

数据模型草案（表结构要点）

- wiki_page：id、parent_id、title、slug、current_revision_id、visibility。
- wiki_revision：id、page_id、content_md、created_by、created_at、comment（未指定）。
- wiki_link：from_page_id、to_page_id（可由解析生成）。
- wiki_acl：resource_id、principal（user/group）、permission（read/write/admin）。
- content_attachment：同博客/云盘复用。

安全与权限考虑

- 版本历史可能包含敏感信息；删除/清理策略：未指定。
- 权限必须双层校验：页面可见性 + ACL；搜索结果同样需过滤。
- Markdown 渲染 XSS 防护同博客。 69

运维/部署建议

- 备份：Wiki 属高价值内容，纳入 PITR。 41
- 监控：编辑失败率、全文检索耗时。
- 证书/反向代理：统一 TLS；可对匿名访问路径做缓存。 42
- frp：同博客，公开访问场景要有 WAF/限流。 54

与其他模块的依赖/集成点

- 与云盘附件、全局搜索、审计日志。
- 与 SSH/frp：可将运维 SOP 与配置版本关联（未来扩展，未指定）。

可扩展性与可插拔性建议

- 支持多空间（Space）与空间级 ACL（未指定）。
- 支持导出（静态站点/Markdown bundle：未指定）。

测试要点

- 并发编辑冲突处理（乐观锁/锁定：未指定）。
- 版本回滚正确性。
- 权限过滤：匿名无法通过历史版本读取受限内容。

测试与验收

全站测试策略（建议）

- 单元测试：权限策略、数据模型、配置解析（frp）、token 生成与校验。
- 集成测试：
- API（含 CSRF/session）与前端交互；DRF 在 SessionAuthentication 下对 CSRF 的要求需覆盖测试用例。 70
- WebSocket：SSH 终端、实时日志；反向代理 Upgrade 配置在 staging 环境验证。 71
- 安全测试：
- CSRF、XSS、越权访问、搜索泄露、token 猜测。 72
- 基础安全基线：TLS、HSTS、安全响应头。 73
- 运维演练：
- 备份恢复演练（DB PITR/逻辑恢复）；PostgreSQL 连续归档/PITR 参考官方文档。 41
- 证书续期演练：ACME 客户端与续期失败回退（未指定）。 36

验收标准（示例）

- 权限：访客无法访问任何管理 API；Trusted 仅能访问被授权资源；Owner 拥有全管理能力。
- 可用性：任一外部服务（Minecraft、HFS、code-server）故障不影响门户与博客/Wiki 的基本访问（降级展示）。
- 可恢复性：可从最近备份恢复 DB；云盘/附件数据可校验完整性（校验方法：未指定）。

- 安全：全站 HTTPS；HSTS 生效；关键管理接口具备限流（实现方式：未指定）。 ⁷⁴

估算表格模板

成本假设、带宽单价、存储单价、机器规格：**未指定**。建议上线前先填入当前实际 VPS/家庭宽带参数，再按监控数据迭代校准。 ²⁷

模块/组件	存储 (GB) 估算	带宽 (月 GB) 估算	CPU (vCPU) 估算	内存 (GB) 估算	峰值 并发/ 连接	成本 假设	备注
PostgreSQL (业务数据)	未指定	未指定	未指定	未指定	未指定	未指定	需考虑 PITR/ WAL 额外占用 ⁷⁵
文件存储（云 盘/附件）	未指定	未指定	未指定	未指定	未指定	未指定	需快照/版本 化；临时文件保 留期未指定
反向代理 (Nginx)	很小	与访问 量相关	低	低	未指定	未指定	WS 需 Upgrade 配置 ⁷
Django API (ASGI)	很小	与 API 调用相 关	未指定	未指定	未指定	未指定	Uvicorn 运行参 考 ⁸
Redis（可选）	很小	很小	低	低	WS 连接 数	未指定	用于 channel layer/队列 ⁷⁶
LibreSpeed 测 速	很小	可能很 高	中	中	未指定	未指定	需要限流 ⁴⁶
Minecraft Server	较大	中/高	中/高	中/高	在线 人数	未指定	最低需求见官方 说明 ⁷⁷
邮件栈（可 选）	中	中	中	中	未指定	未指定	SPF/DKIM/ DMARC/MTA- STS 复杂度高 ⁷⁸
code-server	中	中	中	中	未指定	未指定	需 WS 代理 ⁷⁹

优先参考资料链接清单（官方/权威）

说明：以下以“优先参考”为原则列出。链接为便于开发/运维团队直接查阅；具体实现仍需结合你的实际部署条件（未指定）。

Fluent 2 Design System（设计规范/Token）

- <https://fluent2.microsoft.design/>
- <https://fluent2.microsoft.design/design-tokens>
- <https://fluent2.microsoft.design/typography>
- <https://fluent2.microsoft.design/layout>
- Fluent UI Web Components: <https://learn.microsoft.com/en-us/fluent-ui/web-components/>
- Fluent UI Web Components Storybook: <https://storybooks.fluentui.dev/web-components/>

Django（中文/英文官方文档）

- Django 6.0 中文目录: <https://docs.djangoproject.com/zh-hans/6.0/contents/>
- Django 认证系统: <https://docs.djangoproject.com/en/6.0/topics/auth/default/>
- Django 安全: <https://docs.djangoproject.com/en/6.0/topics/security/>
- Django CSRF: <https://docs.djangoproject.com/en/6.0/ref/csrf/>
- Django ASGI + Uvicorn: <https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/uvicorn/>

Django REST framework（API 认证/权限）

- <https://www.django-rest-framework.org/api-guide/authentication/>
- <https://www.django-rest-framework.org/api-guide/permissions/>

PostgreSQL（备份/PITR/全文检索）

- Backup & Restore: <https://www.postgresql.org/docs/current/backup.html>
- Continuous Archiving & PITR: <https://www.postgresql.org/docs/current/continuous-archiving.html>
- Full Text Search types: <https://www.postgresql.org/docs/current/datatype-textsearch.html>
- Django Postgres Search: <https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/search/>

Nginx 反向代理 WebSocket

- <https://nginx.org/en/docs/http/websocket.html>

Let's Encrypt / ACME

- <https://letsencrypt.org/ja/docs/client-options/>

frp（官方）

- <https://github.com/fatedier/frp>
- <https://gofrp.org/en/docs/features/common/configure/>

LibreSpeed（官方）

- <https://librespeed.org/>
- <https://github.com/librespeed/speedtest/blob/master/doc.md>

HFS（官方）

- <https://rejetto.com/hfs/>
- https://rejetto.com/wiki/index.php?title=HFS:_Introduction.html

code-server（官方）

- <https://github.com/coder/code-server/blob/main/docs/guide.md>

Minecraft (官方)

- EULA: <https://www.minecraft.net/en-us/eula>
- Setup Java Server: <https://help.minecraft.net/hc/en-us/articles/360058525452-How-to-Setup-a-Minecraft-Java-Edition-Server>
- Dedicated Server System Requirements: <https://edusupport.minecraft.net/hc/en-us/articles/46913335157140-Dedicated-Server-System-Requirements>

邮件安全 (权威)

- Postfix TLS: https://www.postfix.org/TLS_README.html
- Dovecot SSL/TLS: <https://doc.dovecot.org/main/core/config/ssl.html>
- DMARC 导入指南 (日文权威 PDF) : https://www.dekyo.or.jp/soudan/data/anti_spam/dmarc_guideline.pdf
- MTA-STS RFC 8461: <https://datatracker.ietf.org/doc/html/rfc8461>

安全最佳实践 (权威)

- OWASP TLS Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html
- OWASP HSTS Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html
- OWASP HTTP Security Headers Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers_Cheat_Sheet.html

可观测性 (官方)

- Prometheus Exporters: <https://prometheus.io/docs/instrumenting/exporters/>
- postgres_exporter: https://github.com/prometheus-community/postgres_exporter
- Loki docs: <https://grafana.com/docs/loki/latest/>

1 5 68 Design tokens - Fluent 2 Design System

https://fluent2.microsoft.design/design-tokens?utm_source=chatgpt.com

2 6 7 71 79 WebSocket proxying

https://nginx.org/en/docs/http/websocket.html?utm_source=chatgpt.com

3 Transport Layer Security - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html?utm_source=chatgpt.com

4 25 Compose Specification (仕様) - Docker ドキュメント

https://docs.docker.jp/compose/compose-file/?utm_source=chatgpt.com

8 How to use Django with Uvicorn

https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/uvicorn/?utm_source=chatgpt.com

9 57 Django Channels — Channels 4.3.2 documentation

https://channels.readthedocs.io/?utm_source=chatgpt.com

10 状态管理

https://cn.vuejs.org/guide/scaling-up/state-management?utm_source=chatgpt.com

11 Fluent UI Web Components Overview

https://learn.microsoft.com/en-us/fluent-ui/web-components/?utm_source=chatgpt.com

12 Using the Django authentication system

https://docs.djangoproject.com/en/6.0/topics/auth/default/?utm_source=chatgpt.com

13 40 72 Cross Site Request Forgery protection

https://docs.djangoproject.com/en/6.0/ref/csrf/?utm_source=chatgpt.com

14 65 OpenID Connect - django-allauth

https://docs.allauth.org/en/dev/socialaccount/providers/openid_connect.html?utm_source=chatgpt.com

15 Permissions

https://www.django-rest-framework.org/api-guide/permissions/?utm_source=chatgpt.com

16 Documentation: 18: 8.11. Text Search Types

https://www.postgresql.org/docs/current/datatype-textsearch.html?utm_source=chatgpt.com

17 PgBouncer command-line usage

https://www.pgbouncer.org/usage.html?utm_source=chatgpt.com

18 76 Channel Layers — Channels 4.3.2 documentation

https://channels.readthedocs.io/en/latest/topics/channel_layers.html?utm_source=chatgpt.com

19 38 PostgreSQL: Documentation: 9.1: Backup and Restore

https://www.postgresql.org/docs/9.1/backup.html?utm_source=chatgpt.com

20 41 75 25.3. Continuous Archiving and Point-in-Time Recovery ...

https://www.postgresql.org/docs/current/continuous-archiving.html?utm_source=chatgpt.com

21 pgBackRest User Guide - Debian & Ubuntu

https://pgbackrest.org/user-guide.html?utm_source=chatgpt.com

22 27 Exporters and integrations

https://prometheus.io/docs/instrumenting/exporters/?utm_source=chatgpt.com

23 A PostgreSQL metric exporter for Prometheus ...

https://github.com/prometheus-community/postgres_exporter?utm_source=chatgpt.com

24 53 Loki overview | Grafana Loki documentation

https://grafana.com/docs/loki/latest/get-started/overview/?utm_source=chatgpt.com

26 7.12 HEALTHCHECK 健康检查| Docker 从入门到实践

https://yeasy.gitbook.io/docker_practice/di-er-bu-fen-jin-jie-pian/07_dockerfile/7.12_healthcheck?utm_source=chatgpt.com

28 37 49 73 74 HTTP Strict Transport Security Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html?utm_source=chatgpt.com

29 HTTP Security Response Headers Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers-Cheat-Sheet.html?utm_source=chatgpt.com

30 46 47 54 Rate limiting rules • Cloudflare Web Application Firewall ...

https://developers.cloudflare.com/waf/rate-limiting-rules/?utm_source=chatgpt.com

31 NGINX ModSecurity WAF

https://docs.nginx.com/nginx/admin-guide/dynamic-modules/nginx-waf/?utm_source=chatgpt.com

32 Proactive DDoS defense

https://developers.cloudflare.com/ddos-protection/best-practices/proactive-defense/?utm_source=chatgpt.com

33 78 送信ドメイン認証技術 DMARC 導入ガイドライン

https://www.dekyo.or.jp/soudan/data/anti_spam/dmarc_guideline.pdf?utm_source=chatgpt.com

- 34 64 **RFC 8461 - SMTP MTA Strict Transport Security (MTA-STS)**
https://datatracker.ietf.org/doc/html/rfc8461?utm_source=chatgpt.com
- 35 58 **SSH Security Best Practices: Protect Your Remote Access ...**
https://tailscale.com/learn/ssh-security-best-practices-protecting-your-remote-access-infrastructure?utm_source=chatgpt.com
- 36 42 **ACME クライアント実装**
https://letsencrypt.org/ja/docs/client-options/?utm_source=chatgpt.com
- 39 43 51 **fatedier/frp: A fast reverse proxy to help you expose a local ...**
https://github.com/fatedier/frp?utm_source=chatgpt.com
- 44 45 **HFS ~ HTTP File Server**
https://rejetto.com/hfs/?utm_source=chatgpt.com
- 48 **librespeed/speedtest**
https://github.com/librespeed/speedtest?utm_source=chatgpt.com
- 50 **speedtest/doc.md at master • librespeed ...**
https://github.com/librespeed/speedtest/blob/master/doc.md?utm_source=chatgpt.com
- 52 **Configuration File - frp**
https://gofrp.org/en/docs/features/common/configure/?utm_source=chatgpt.com
- 55 **frp/conf/frps_full_example.toml at dev • fatedier/frp**
https://github.com/fatedier/frp/blob/dev/conf/frps_full_example.toml?utm_source=chatgpt.com
- 56 **Documentation**
https://xtermjs.org/docs/?utm_source=chatgpt.com
- 59 **Protect SSH with Fail2ban on Ubuntu: Tutorial**
https://www.digitalocean.com/community/tutorials/how-to-protect-ssh-with-fail2ban-on-ubuntu-20-04?utm_source=chatgpt.com
- 60 61 **User License Agreement (“EULA”)**
https://www.minecraft.net/en-us/eula?utm_source=chatgpt.com
- 62 63 **Postfix TLS Support**
https://www.postfix.org/TLS_README.html?utm_source=chatgpt.com
- 66 **code-server/docs/guide.md at main**
https://github.com/coder/code-server/blob/main/docs/guide.md?utm_source=chatgpt.com
- 67 69 **Security in Django**
https://docs.djangoproject.com/en/6.0/topics/security/?utm_source=chatgpt.com
- 70 **Authentication**
https://www.django-rest-framework.org/api-guide/authentication/?utm_source=chatgpt.com
- 77 **Dedicated Server System Requirements**
https://edusupport.minecraft.net/hc/en-us/articles/46913335157140-Dedicated-Server-System-Requirements?utm_source=chatgpt.com